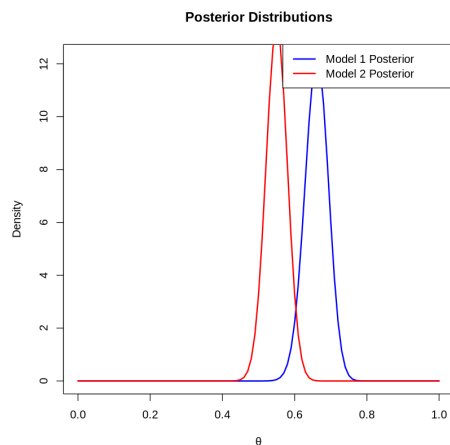


Part 1

```
1 #Given data
2 y <- c(10, 15, 15, 14, 14, 14, 13, 11, 12, 16)
3 n <- 20
4 N <- length(y)
5 sum_y <- sum(y)
```

Exercise 1.1

```
1 # --- Exercise 1.1: Graph the posterior distribution ---
2 # Calculate exact posterior parameters
3 a_post1 <- 6 + sum(y)
4 b_post1 <- 6 + sum(n - y)
5 a_post2 <- 20 + sum(y)
6 b_post2 <- 60 + sum(n - y)
7
8 # Plotting
9 curve(dbeta(x, a_post1, b_post1), from=0, to=1, col="blue", lwd=2,
10       ylab="Density", xlab=expression(theta), main="Posterior Distributions")
11 curve(dbeta(x, a_post2, b_post2), from=0, to=1, col="red", lwd=2, add=TRUE)
12 legend("topright", legend=c("Model 1 Posterior", "Model 2 Posterior"),
13       col=c("blue", "red"), lwd=2)
```



Exercise 1.2

```
1 set.seed(123)
2 theta_samples1 <- rbeta(S, a_post1, b_post1)
3 theta_samples2 <- rbeta(S, a_post2, b_post2)
4
5 calc_lppd <- function(data, trials, theta_samples) {
6   lpd_i <- sapply(data, function(yi) {
7     log(mean(dbinom(yi, size=trials, prob=theta_samples)))
8   })
9   return(sum(lpd_i))
10 }
11
12 lppd_m1 <- calc_lppd(y, n, theta_samples1)
13 lppd_m2 <- calc_lppd(y, n, theta_samples2)
14
15 cat("Model 1 lppd:", lppd_m1)
16 cat("\nModel 2 lppd:", lppd_m2)
```

```
Model 1 lppd: -20.36967
Model 2 lppd: -25.84488
```

Exercise 1.3

```
1 dev_m1 <- -2 * lppd_m1
2 dev_m2 <- -2 * lppd_m2
3
4 cat("Model 1 In-sample Deviance:", dev_m1)
5 cat("\nModel 2 In-sample Deviance:", dev_m2)
```

```
Model 1 In-sample Deviance: 40.73934
Model 2 In-sample Deviance: 51.68976
```

It is called in-sample deviance because we are evaluating the model's predictive accuracy using the same dataset (y) that we used to fit the model and form the posterior distribution. This typically results in an overly optimistic evaluation of the model because the model has already seen and adapted to this data.

Exercise 1.4

Based on in-sample deviance, Model 1 is a better fit for the data. Its less restrictive prior allowed the posterior to closely center around the data's true mean resulting in lower deviance.

Exercise 1.5

```
1 y_new <- c(5, 6, 10, 8, 9)
2 lppd_new_m1 <- calc_lppd(y_new, n, theta_samples1)
```

```
3 lppd_new_m2 <- calc_lppd(y_new, n, theta_samples2)
4
5 out_dev_m1 <- -2 * lppd_new_m1
6 out_dev_m2 <- -2 * lppd_new_m2
7
8 cat("Model 1 Out-of-sample Deviance:", out_dev_m1)
9 cat("\nModel 2 Out-of-sample Deviance:", out_dev_m2)
```

Model 1 Out-of-sample Deviance: 50.48496
Model 2 Out-of-sample Deviance: 31.64187

For predicting the new data, Model 2 is actually better. The new data points have a much lower average. Model 2's strong prior pulled its posterior mean down towards 0.55, making it closer to the new dataset than Model 1. This illustrates how a model that fits training data perfectly can sometimes predict different out-of-sample data poorly.

Exercise 1.6

```
1 loo_cv <- function(data, trials, a_prior, b_prior, S=10000) {
2   lpd_loo <- numeric(length(data))
3
4   for(i in 1:length(data)) {
5     y_train <- data[-i]
6     y_test <- data[i]
7
8     # Calculate posterior parameters leaving one out
9     a_post <- a_prior + sum(y_train)
10    b_post <- b_prior + sum(trials - y_train)
11
12    # Sample from the leave-one-out posterior
13    theta_s <- rbeta(S, a_post, b_post)
14
15    # Calculate predictive density for the held-out point
16    lpd_loo[i] <- log(mean(dbinom(y_test, size=trials, prob=theta_s)))
17  }
18  return(sum(lpd_loo))
19 }
20
21 loo_lppd_m1 <- loo_cv(y, n, 6, 6)
22 loo_lppd_m2 <- loo_cv(y, n, 20, 60)
23
24 cat("Model 1 LOO Deviance:", -2 * loo_lppd_m1)
25 cat("\nModel 2 LOO Deviance:", -2 * loo_lppd_m2)
```

Model 1 LOO Deviance: 42.22336
Model 2 LOO Deviance: 54.44836

Part 2

Exercise 2.1

```
1 ML_binomial <- function(k,n,a,b){
2 ML <- (gamma(n+1)/(gamma(k+1)*gamma(n-k+1)))*( #Used Gamma function because of fractional values whose factorial was to be calculated.
3 gamma(k+a-1+1)*gamma(n-k+b-1+1)/gamma(n+a+b-1+1))
4 ML
5 }
6
7 k <- 2
8 n <- 10
```

1 #Beta(0.1,0.4)
2 ML_binomial(k,n,0.1,0.4)

0.473956366844962

1 #Beta(1,1)
2 ML_binomial(k,n,1,1)

0.0909090909090909

1 #Beta(2,6)
2 ML_binomial(k,n,2,6)

0.00472689075630252

1 #Beta(6,2)
2 ML_binomial(k,n,6,2)

0.000231386260798025

1 #Beta(20,60)
2 ML_binomial(k,n,20,60)

5.07939675334424e-21

1 #Beta(60,20)
2 ML_binomial(k,n,60,20)

1.50663033900523e-23

Exercise 2.2

```
1 priors <- list(
2   c(0.1, 0.4),
3   c(1, 1),
```

```
4   c(2, 6),
5   c(6, 2),
6   c(20, 60),
7   c(60, 20)
8 )
9
10 MC_integration <- function(k, n, a, b, S=100000) {
11   theta_s <- rbeta(S, a, b)
12   return(mean(dbinom(k, size=n, prob=theta_s)))
13 }
14
15 set.seed(123)
16 mc_ml_results <- sapply(priors, function(p) MC_integration(k, n, p[1], p[2]))
17
18 print(mc_ml_results)
```

```
[1] 0.0398041725 0.0906857755 0.1985172489 0.0096235500 0.2693406840
[6] 0.0008027328
```